Divide and Conquer

Divide and Conquer

The **divide-and-conquer** paradigm,

Divide the problem into one or more subproblems that are smaller instances of the same problem.

Conquer the subproblems by solving them recursively.

Base case: If the subproblems are small enough, just solve them by brute force.

Combine the subproblem solutions to form a solution to the original problem.

Closest Pair Problem

Input: A set of points in a plane

Output: A pair of points that are closest i.e, the distance between these two points is minimum.

Closest Pair Problem

Input: A set of points in a plane

Output: A pair of points that are closest i.e, the distance between these two points is minimum.

Any algorithm?

Closest Pair Problem

Input: A set of points in a plane

Output: A pair of points that are closest i.e, the distance between these two points is minimum.

Any algorithm?

There are $O(n^2)$ pairs of points and we calculate the distance between each pair of points and return the minimum distance.
$O(n^2)$ time

Closest Pair Problem

Input: A set of points in a plane

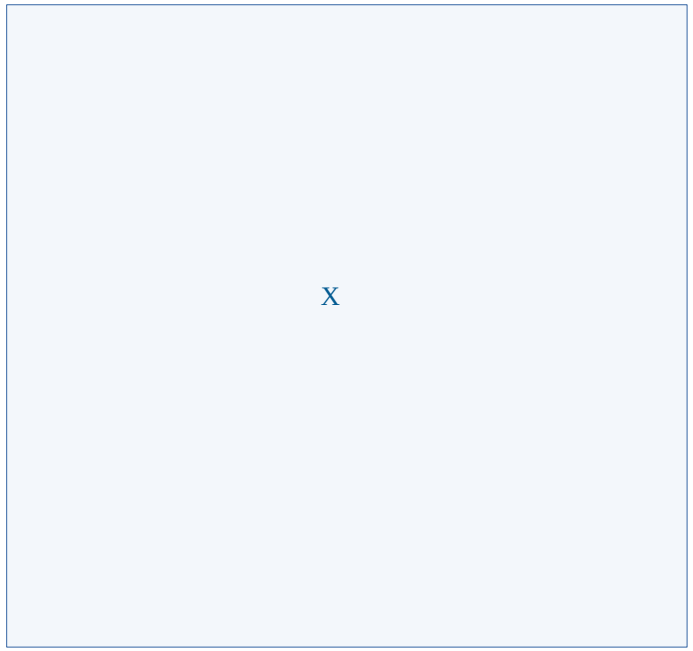
Output: A pair of points that are closest i.e, the distance between these two points is minimum.

Question: Do we need to calculate the distances between every pair of points?

Any algorithm?

There are $O(n^2)$ pairs of points and we calculate the distance between each pair of points and return the minimum distance.
$O(n^2)$ time

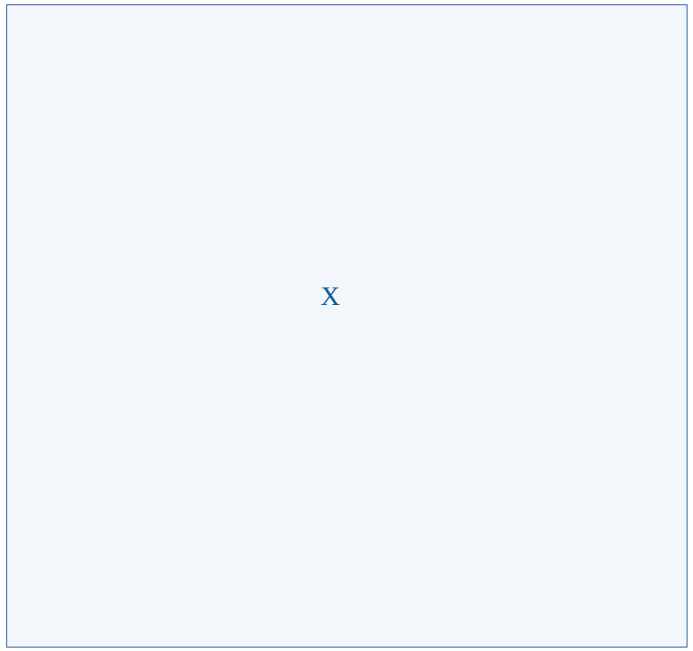
Intuition

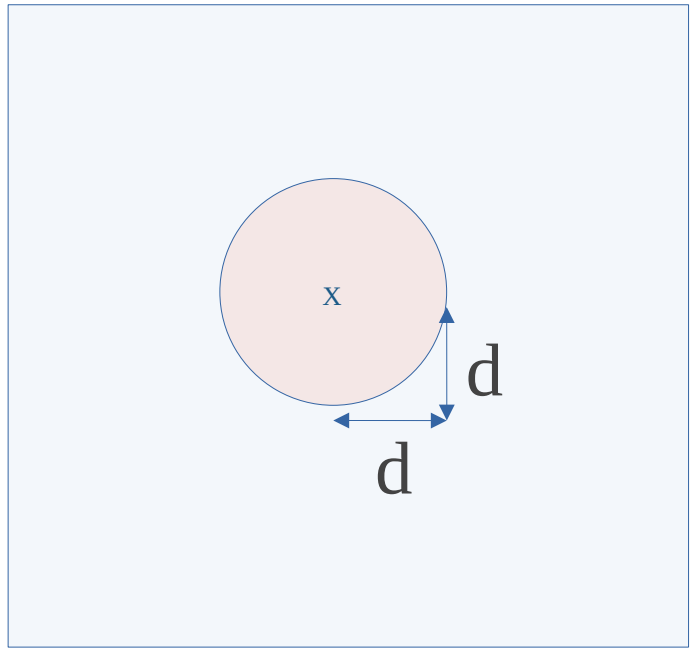


Suppose I have an OPT solution, say d, for P-x.

Do I have to look at all distances with x?

Intuition





Suppose I have an OPT solution, say d, for P-x.

For OPT(P), do I have to look at all distances with x? Not necessarily

1-D closest pair

Input: A set of points on a line

Output: A pair of points that are closest i.e, the distance between these two points is minimum.

1-D closest pair

Input: A set of points on a line

Output: A pair of points that are closest i.e, the distance between these two points is minimum.

Question: Do we need to calculate the distances between every pair of points?

There are $O(n^2)$ pairs of points and we calculate the distance between each pair of points and return the minimum distance.
$O(n^2)$ time

Intuition



- Sort according to their coordinate.
- Calculate distances only for 2n pairs of points
- Choose the minimum

  - **O(n log n) time**

Intuition



- Sort according to their coordinate.
- Calculate distances only for 2n pairs of points
- Choose the minimum

  - **O(n log n) time**

Use a D&C approach!

Closest Pair Problem (1-D)

What about the size of the sub-problem (s)

- One sub-problem

- Two sub-problems

Closest Pair Problem (1-D)

What about the size of the sub-problem (s)

- One sub-problem

$$T(n) = T(n-1) + n-1$$

- Two sub-problems

Closest Pair Problem (1-D)

What about the size of the sub-problem (s)

- One sub-problem

$$T(n) = T(n-1) + n-1$$

- Two sub-problems

$$T(n) = 2T(n/2) + C(n) + D(n)$$

Closest Pair Problem (1-D)

What should the divide and combine step be?

What about the size of the sub-problem (s)

- One sub-problem

$$T(n) = T(n-1) + n-1$$

- Two sub-problems

$$T(n) = 2T(n/2) + C(n) + D(n)$$

Closest Pair Problem (1-D)

What should the divide and combine step be?

Finding median and dividing subproblems + distances of adj. points

Can we adapt the strategy to 2D input?

What about the size of the sub-problem (s)

- One sub-problem

$T(n) = T(n-1) + n-1$

$O(n \log n)$

- Two sub-problems

$T(n) = 2T(n/2) + C(n) + D(n)$

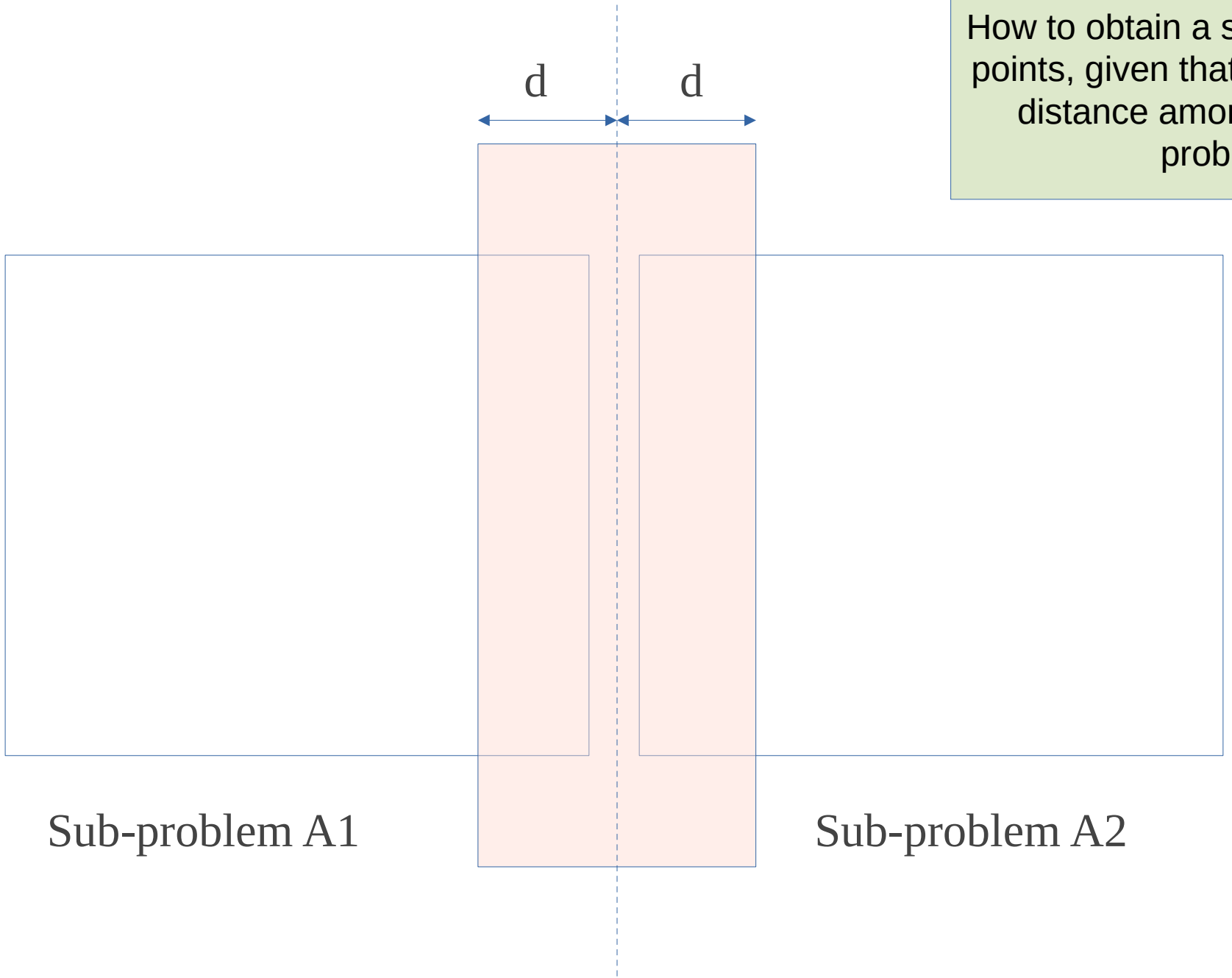
Closest Pair Problem

Input: A set of points in a plane

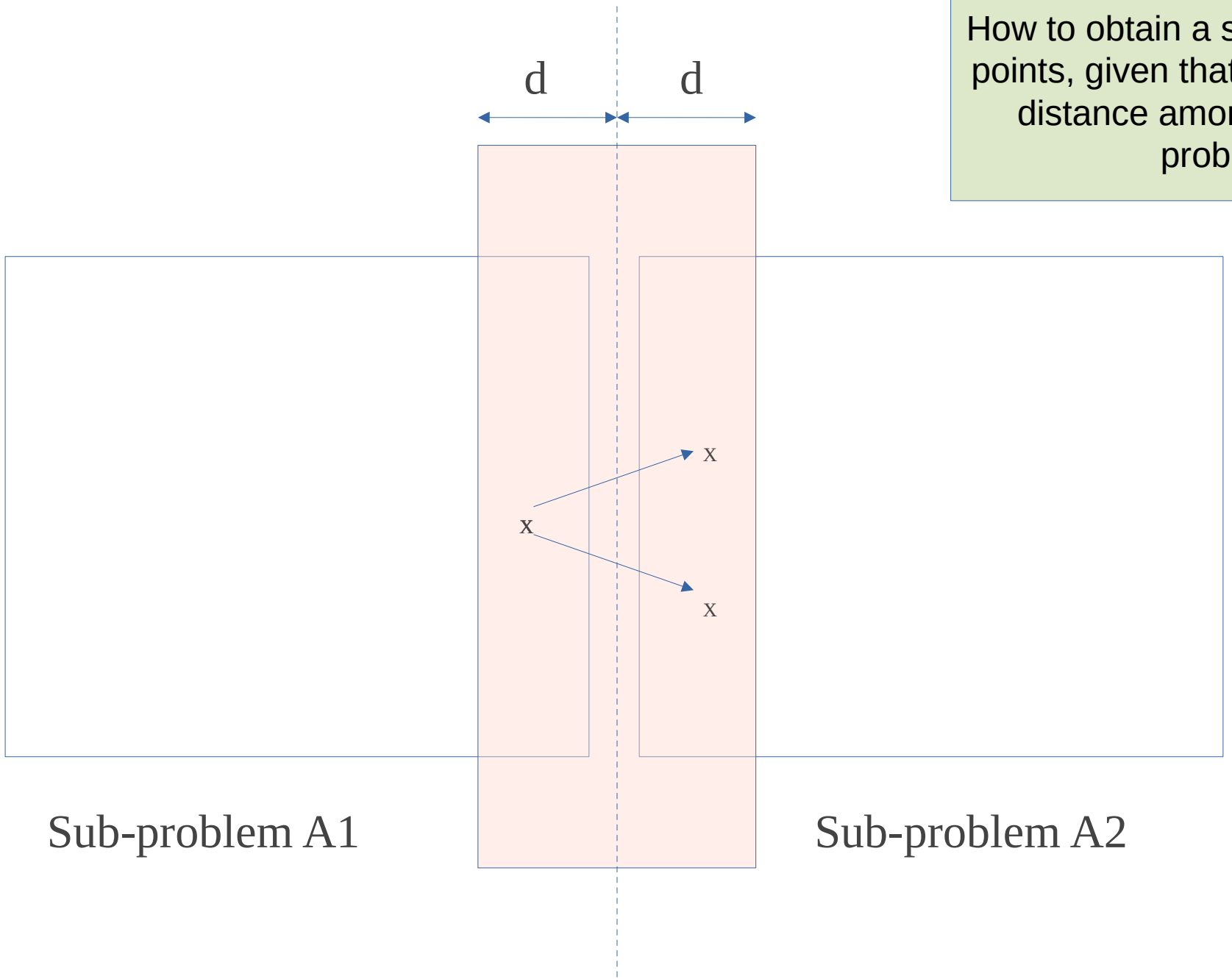
Output: A pair of points that are closest i.e, the distance between these two points is minimum.

Use a D&C approach!

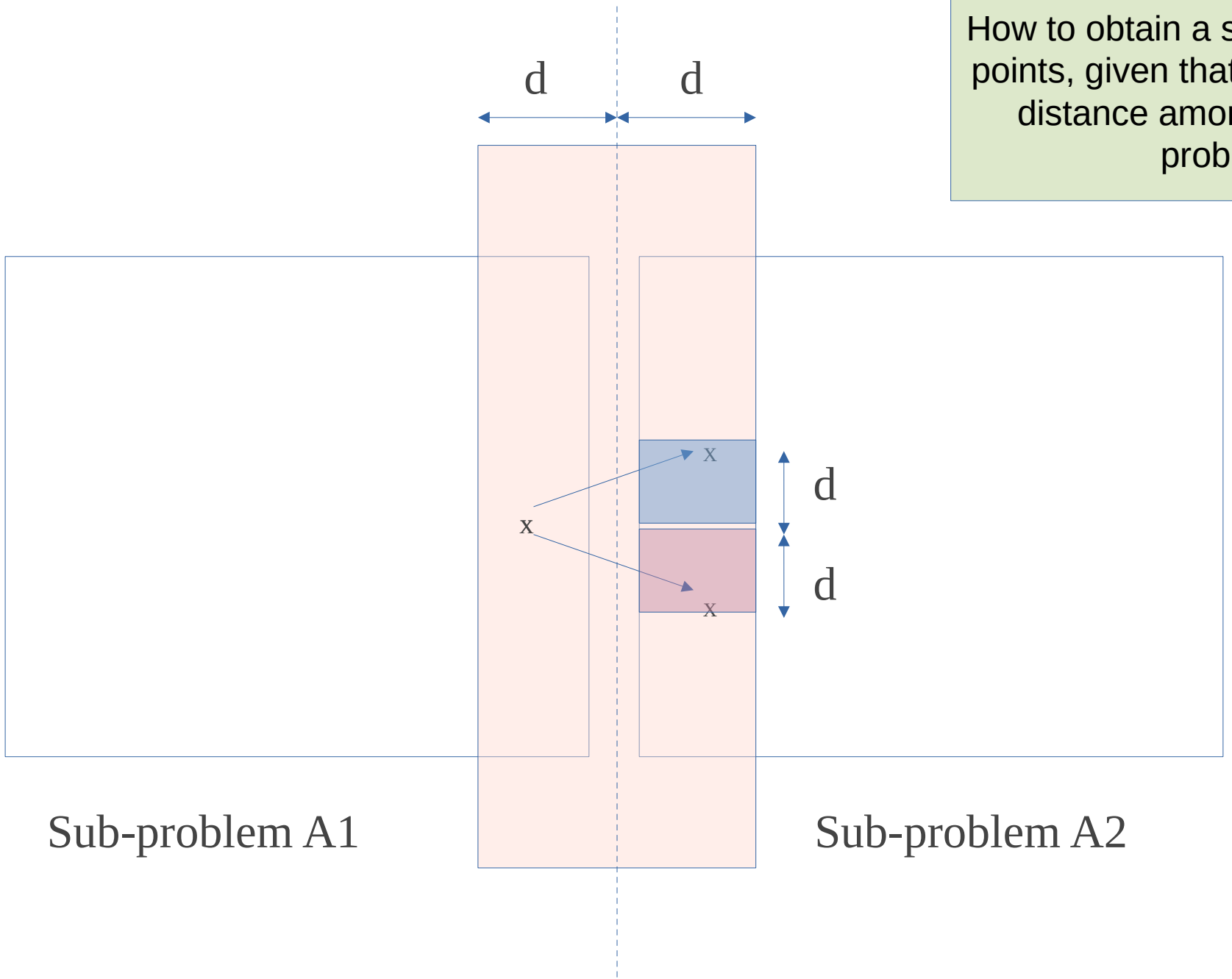
How to obtain a solution from these points, given that d is the minimum distance among the two sub-problems?



How to obtain a solution from these points, given that d is the minimum distance among the two sub-problems?

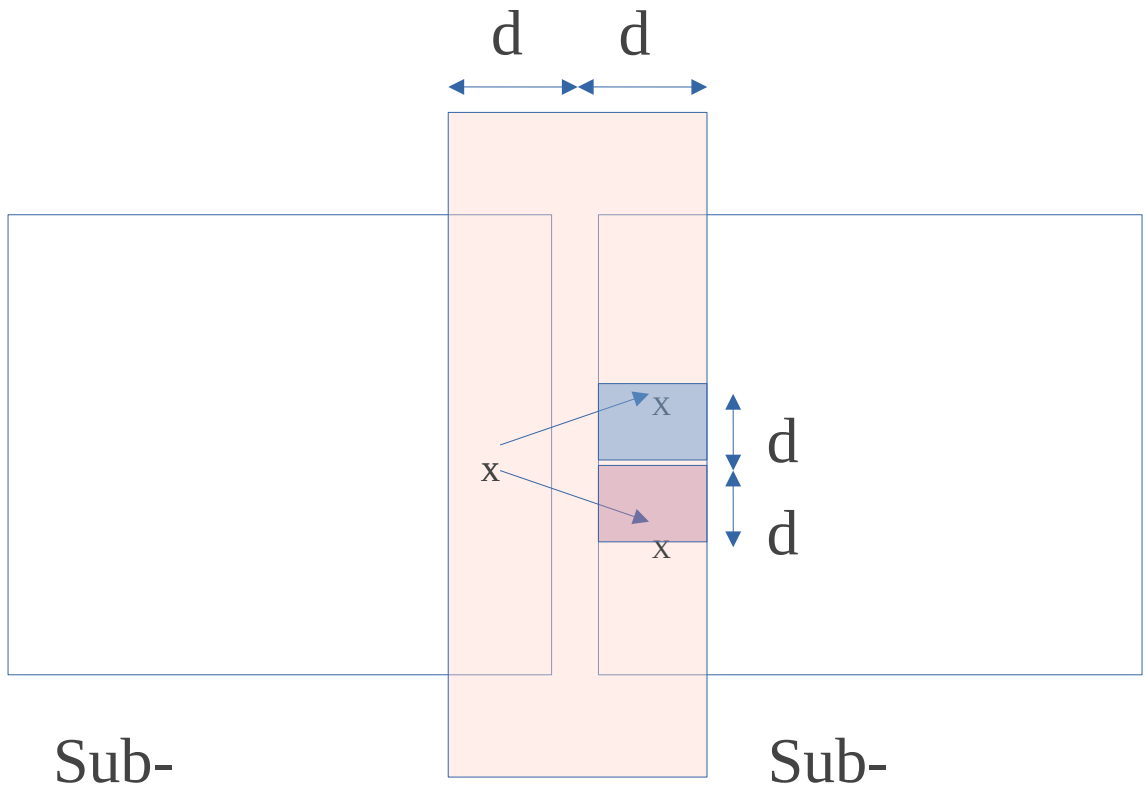


How to obtain a solution from these points, given that d is the minimum distance among the two sub-problems?



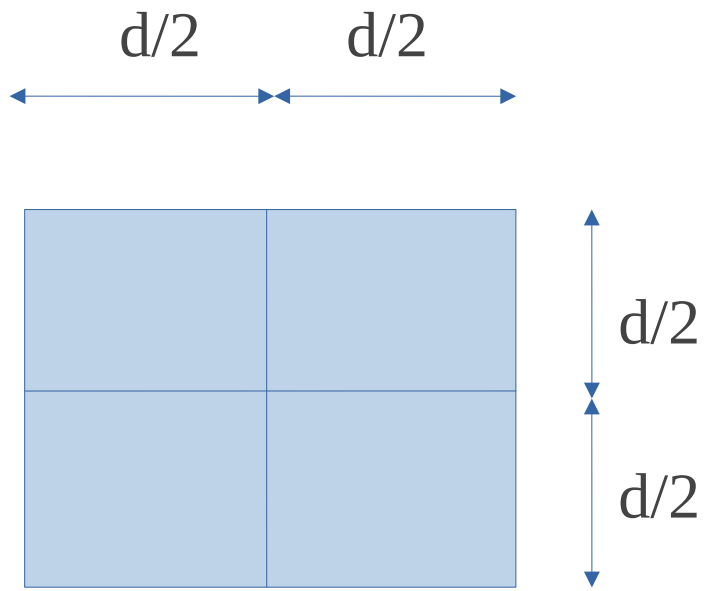
Observation

Number of points in a d*d square **is**



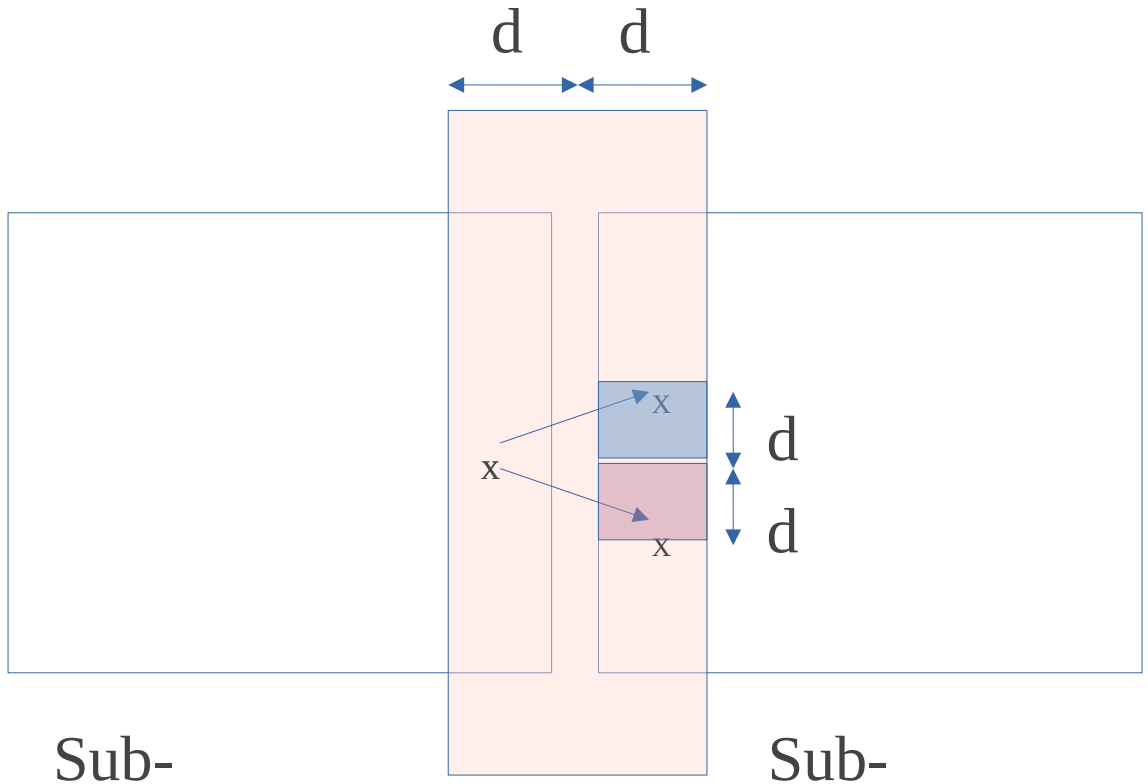
Sub-problem A1

Sub-problem A2



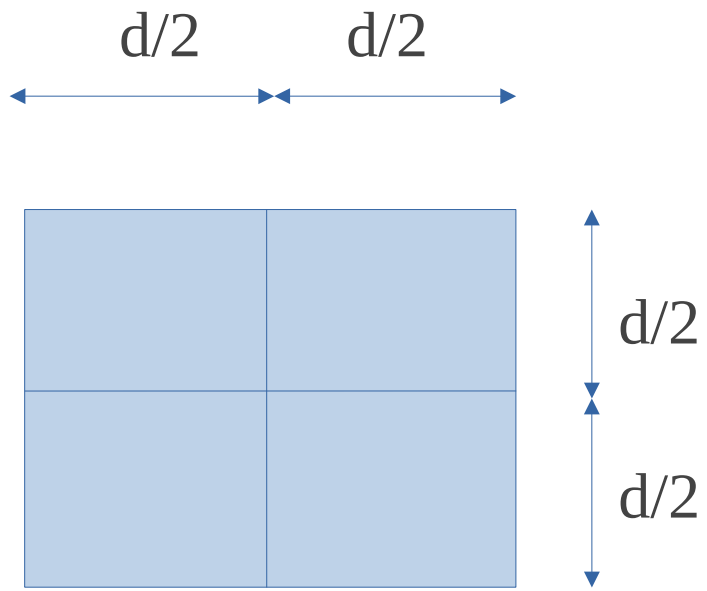
Observation

Number of points in a d*d square **is at most 4**.



Sub-problem A1

Sub-problem A2



How do you find these at most 7 interesting points?

Algorithm

1. Sort the points w.r.t. to X-coordinates (A_X), and w.r.t. to Y-coordinates (A_Y).

2. ALG(A_X, A_Y)
    1. Find the median w.r.t. A_X.
    2. Recurse on the subproblems (A_{X1}, A_{Y1}) and (A_{X2}, A_{Y2})
    3. Combine step: find minimum distance of a pair of points in the strip with one being on the left and the other on the right.

Algorithm

1. Sort the points w.r.t. to X-coordinates (A_X), and w.r.t. to Y-coordinates (A_Y).

2. ALG(A_X, A_Y)
    1. Find the median w.r.t. A_X.
    2. Recurse on the subproblems (A_{X1}, A_{Y1}) and (A_{X2}, A_{Y2})
    3. Combine step: find minimum distance of a pair of points in the strip with one being on the left and the other on the right.

$$T(n)=2T(n/2)+C(n)+D(n)$$

Combine Step

For each point on the left strip, maintain a window of points from the right strip that are within the range.

Keep moving upwards accordingly as we access more points from the left strip.

$T(n)= 2T(n/2)+ O(n) =$ **$O(n \log n)$**